AI-Powered HR Assistant for Nestle

Contents

1.- Architecture 3

2.- Import essential tools and set up OpenAI's API environment 3

3.- Load Nestle's HR policy using PyPDFLoader and split it for easy processing..... 4

5.- Build a question-answering system using the GPT-3.5 Turbo model to retrieve answers from text chunks 5

6.- Create a prompt template to guide the chatbot in understanding and responding to users 6

.- Use Gradio to build a user-friendly chatbot interface, enabling interaction and information retrieval 7

1.- Architecture

Two notebooks are created:

- nestle_ai_project.ipynb: Loads Nestle'HR pdf policy , split the loaded text into chunks and embeds the chunk text into a local chroma vector db. This data base is saved in google drive and will be used by nestle_ai_project_ui notebook.
- nestle_ai_project_ui.ipynb: Loads the saved chroma vector db, creates an LLM model using gpt-3.5-turbo, creates a retriever object from the loaded data base vector, creates a prompt template that uses the context from the database data and the question from the user and finally creates a web interface that answer questions using context and user input.

2.- Import essential tools and set up OpenAI's API environment

nestle_ai_project.ipynb

Install required libraries:

```
gradio
langchain-community==0.4.1
langchain-core==1.1.0
langchain-openai==1.1.0
chromadb==1.3.5
openai==2.8.1
tiktoken==0.12.0
pypdf
langchain-text-splitters==1.0.0
tiktoken
```

```
!pip install -r requirements_backend.txt
```

```
Collecting opentelemetry-sdk>=1.2.0 (from chromadb==1.3.5->-r requirements_backend.txt (line 5))
  Downloading opentelemetry_sdk-1.40.0-py3-none-any.whl.metadata (1.6 kB)
Collecting opentelemetry-api>=1.2.0 (from chromadb==1.3.5->-r requirements_backend.txt (line 5))
  Downloading opentelemetry_api-1.40.0-py3-none-any.whl.metadata (1.5 kB)
Collecting opentelemetry-semantic-conventions==0.61b0 (from opentelemetry-sdk>=1.2.0->chromadb==1
  Downloading opentelemetry_semantic_conventions-0.61b0-py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pand
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pa
Collecting backoff>=1.10.0 (from posthog<6.0.0,>=2.4.0->chromadb==1.3.5->-r requirements_backend.
  Downloading backoff-2.2.1-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (
```

Import needed libraries

```
[1]
from langchain_community.document_loaders import TextLoader, PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
```

3.- Load Nestle's HR policy using PyPDFLoader and split it for easy processing

nestle_ai_project.ipynb

Load PDF

```
pdf_loader = PyPDFLoader("1728286846_the_nestle_hr_policy_pdf_2012.pdf")
pdf_pages = pdf_loader.load_and_split()
print(pdf_pages[0])  #
```

```
page_content='Policy
Mandatory
September   2012
The Nestlé
Human Resources Policy' metadata={'producer': 'Adobe PDF Library 10.0.1', 'creator': 'Adobe InDes
```

Split the content in chunks.

```
(function) chunk_overlap: Any
doc_splitter = RecursiveCharacterTextSplitter(chunk_size=1024, chunk_overlap=64)
split_texts = doc_splitter.split_documents(pdf_pages)
print(len(split_texts))  # Prints the number of chunks the PDF has been split into

21
```

4.- Create vector representations for text chunks using Chroma dB and OpenAI's embeddings

nestle_ai_project.ipynb

Create ChomeDB in google drive

```
[5]
3s
openai_embed = OpenAIEmbeddings(api_key="sk-proj-eIwtDvIacKR14ovPvVxs9rJevV-3lGyGt2W5qM4SxfkRV27sgCNoVTiTn1Useu
openai_embed_result = openai_embed.embed_documents([text.page_content for text in split_texts])
print(len(openai_embed_result[0]))
print(len(openai_embed_result))

/tmp/ipykernel_2027/2547387568.py:1: LangChainDeprecationWarning: The class `OpenAIEmbeddings` was deprecated in
  openai_embed = OpenAIEmbeddings(api_key="sk-proj-eIwtDvIacKR14ovPvVxs9rJevV-3lGyGt2W5qM4SxfkRV27sgCNoVTiTn1Use
1536
21

[6]
5s
persist_directory = "/content/drive/MyDrive/Colab LLM Project/chroma_db"

vector_db = Chroma.from_documents(
            documents=split_texts,
            embedding=openai_embed,
            persist_directory=persist_directory
        )
```

5.- Build a question-answering system using the GPT-3.5 Turbo model to retrieve answers from text chunks

nestle_ai_project_ui.ipynb

```
[1]
22s
import gradio as gr
from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```
[2]
2s
persist_directory = "/content/drive/MyDrive/Colab LLM Project/chroma_db"

openai_embed = OpenAIEmbeddings(api_key="sk-proj-eIwtDvIacKR14ovPvVxs9rJevV-3lGyGt2W5qM4SxfkRV27sgCNoVTiTn1UseuDQjikh9it5ifT3Bl

2s
vector_db = Chroma(
    persist_directory=persist_directory,
    embedding_function=openai_embed
)

llm = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0,
    api_key="sk-proj-eIwtDvIacKR14ovPvVxs9rJevV-3lGyGt2W5qM4SxfkRV27sgCNoVTiTn1UseuDQjikh9it5ifT3BlbkFJrRjq3hmZ9EK4mcQdsV4x0uC
)

retriever = vector_db.as_retriever(search_kwargs={"k": 3})

prompt = ChatPromptTemplate.from_template(
"""
Answer the question using the context below.

Context:
{context}

Question:
{input}
"""
)
```

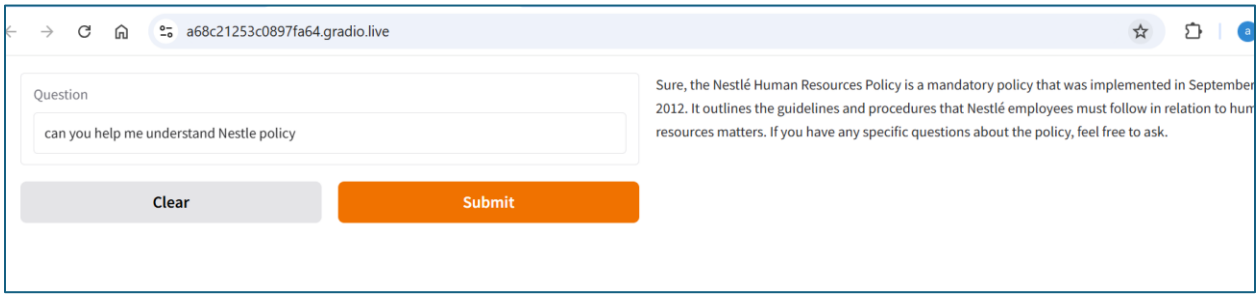
6.- Create a prompt template to guide the chatbot in understanding and responding to users

nestle_ai_project_ui.ipynb

```
prompt = ChatPromptTemplate.from_template(
"""
Answer the question using the context below.

Context:
{context}

Question:
{input}
"""
)
```

.- Use Gradio to build a user-friendly chatbot interface, enabling interaction and information retrieval

nestle_ai_project_ui.ipynb

```
def fnQuery(query):
  docs = retriever.invoke(query)
  context = "\n\n".join(doc.page_content for doc in docs)
  formatted_prompt = prompt.format(
                     context=context,
                     input=query
                     )
  response = llm.invoke(formatted_prompt)
  return response.content

demo = gr.Interface(fn=fnQuery, inputs=gr.Textbox(label="Question"), outputs=gr.Markdown(), flagging_mode="never")
demo.launch()
```

```
It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True`

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://a68c21253c0897fa64.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the workin
```